

Candle Science Engine – Requirements & Design Document

Version: 17.5
Platform: TradingView / Pine Script v6
Last Updated: February 2026

1. PURPOSE

The Candle Science Engine is a TradingView overlay indicator that predicts the probable behavior of the next candle (C3) by analyzing the structural relationship between two preceding candles (C1 and C2). It scans all available chart history for matching C1→C2 patterns and computes statistical distributions of C3 outcomes.

The indicator is instrument- and timeframe-agnostic. Primary use case is NQ futures on daily timeframe but it works on any market.

2. TERMINOLOGY

Term	Definition
C1	The first candle in the 3-candle triplet (oldest)
C2	The second candle (reference candle – the one just completed)
C3	The third candle (the one being predicted)
OHLC	Open, High, Low, Close of any candle
MFE	Maximum Favorable Excursion – how far price moved in the favorable direction beyond a reference level (e.g., C3H above C2H)
MAE	Maximum Adverse Excursion – how far price moved against you from a reference level (e.g., C3L below C2H when you expected it to break up)
Contained	When C3 does NOT break a C2 level (e.g., C3H stays below C2H)
Pattern Dimension	A single boolean structural relationship between candles (e.g., "C2H > C1H")

Term	Definition
Auto-Detect	Mode where the engine reads the current C1→C2 pattern and matches historical triplets automatically
Manual	Mode where the user explicitly specifies which filter criteria to apply

3. CORE ALGORITHM

3.1 Data Collection Phase (runs every bar)

On every bar where `(bar_index >= 4)`, store the raw OHLC of the triplet `([bar-3, bar-2, bar-1])` into 12 persistent arrays:

```
hist_c1h, hist_c1l, hist_c1o, hist_c1c
hist_c2h, hist_c2l, hist_c2o, hist_c2c
hist_c3h, hist_c3l, hist_c3o, hist_c3c
```

CRITICAL ARCHITECTURE NOTE: Data collection happens on every bar, but filtering and statistical computation happen ONLY on `(barstate.islast)`. This is essential because the "current pattern" (`(cur_c1d)`, `(cur_c2h_gt_c1h)`, etc.) changes on every bar – if filtering ran during the bar-by-bar pass, each historical bar would be compared against its own contemporaneous pattern rather than the live chart's pattern. This was a critical bug in versions prior to v17.4.

3.2 Current Pattern Detection (computed every bar, used only on last)

In Real-time mode (`(c1_off=2, c2_off=1)`):

- C1 = bar two bars ago
- C2 = bar one bar ago (last closed)
- C3 open = current bar's open

In Historical mode (`(c1_off=1, c2_off=0)`):

- C1 = bar one bar ago
- C2 = current bar
- C3 open = hypothetical (user-configurable zone within C2 range)

Compute 16 boolean relationships that define the current pattern:

Direction (2):

1. C1 direction (bull/bear)
2. C2 direction (bull/bear)

C2 vs C1 Structure (10): 3. C2H vs C1H (higher-high / lower-high) 4. C2H vs C1O 5. C2L vs C1L (higher-low / lower-low) 6. C2L vs C1O 7. C2C vs C1H 8. C2C vs C1L 9. C2C vs C1C 10. C2C vs C1O 11. C2O vs C1C 12. C2O vs C1O

C3 Open vs C2 (4): 13. C3O vs C2H 14. C3O vs C2L 15. C3O vs C2C 16. C3O vs C2O

3.3 Filter/Scan Phase (runs only on `barstate.islast`)

Reset all counters and result arrays. Loop through all stored historical triplets. For each triplet:

1. Compute the same 16 boolean relationships for the historical C1→C2→C3
2. Compare each against the current pattern using `passes_dir_check()` or `passes_rel_check()`
3. In Auto-Detect mode: dimension passes if it's disabled OR historical value matches current value
4. In Manual mode: dimension passes if filter is "Any" OR historical value matches the specified filter
5. If ALL 16 dimensions pass → triplet is a match

For each match, accumulate:

- Direction counters (bull/bear)
- Breakout counters (C3H>C2H, C3L<C2L)
- Close counters (C3C>C2H, C3C>C2L, C3C>C2C, C3C>C2O)
- 8 excursion arrays (C3H and C3L relative to each of C2H, C2L, C2C, C2O), all as % of C2C
- Body array ((C3C-C3O)/C2C * 100)
- Close array ((C3C-C2C)/C2C * 100)
- 2 conditional (contained) arrays: C3H-C2H when C3H≤C2H, C3L-C2L when C3L≥C2L

3.4 Probability Computation

CRITICAL: All probability divisions must cast to `float()` before dividing. Pine Script v6 performs integer division on `int/int`, which truncates (e.g., $43/96 = 0$ instead of 0.448).

```
float p_bull = float(c3_bull_count) / float(total_filtered) * 100.0
```

Computed probabilities:

- `p_bull` / `p_bear` – C3 direction
- `p_high_gt_c2h` / `p_high_lt_c2h` – C3 high vs C2 high
- `p_low_lt_c2l` / `p_low_gt_c2l` – C3 low vs C2 low
- `p_close_gt_c2h` – C3 close above C2 high
- `p_close_gt_c2l` – C3 close above C2 low
- `p_close_gt_c2c` – C3 close above C2 close
- `p_close_gt_c2o` – C3 close above C2 open

3.5 Percentile Function (`pct_interp`)

Linear interpolation percentile. Given a float array and a percentile p (0-100):

1. Copy and sort ascending
2. Compute rank = $(p/100) \times (n-1)$
3. Interpolate between floor and ceil indices

Used for medians (p=50) and trim boundaries.

4. DISTRIBUTION ENGINE

4.1 DistResult Type

```
type DistResult
    array<float> bounds    // bucket boundaries [lo0, hi0, lo1, hi1, ...]
    array<int>   counts    // count per bucket
    int          n         // total filtered sample count
    float        median    // P50 of filtered values
    float        zone_lo   // auto-zone lower bound
    float        zone_hi   // auto-zone upper bound
    float        zone_pct  // % of samples in auto-zone
```

4.2 `build_dist(src, filter_sign)` Function

Parameters:

- `src`: raw excursion array (all values, positive and negative)
- `filter_sign`: `"pos"` (keep >0), `"neg"` (keep <0), or `"all"`

Steps:

1. Filter source array by sign
2. If $n \geq 2$: compute median, trim boundaries, build buckets, find auto-zone
3. If $n = 1$: set median only
4. Return `DistResult`

Bucketing logic:

- For positive values: buckets go low→high, last bucket gets +0.0001 to include upper bound
- For negative values: buckets go from closest-to-zero downward (display order: least negative first), last bucket gets -0.0001

Auto-zone algorithm: Find the tightest contiguous span of buckets that contains at least `threshold%` of samples. Uses nested loop: for each start bucket, accumulate forward until threshold is met, track minimum span.

CRITICAL: All int divisions in bucket percentage and zone percentage calculations must use `float()` casts.

4.3 Eight Standard Distributions

Variable	Source Array	Filter	Meaning
<code>dist_mfe_c2h</code>	<code>arr_c3h_vs_c2h</code>	<code>pos</code>	C3H excursion above C2H
<code>dist_mae_c2h</code>	<code>arr_c3l_vs_c2h</code>	<code>neg</code>	C3L drop below C2H
<code>dist_mfe_c2l</code>	<code>arr_c3l_vs_c2l</code>	<code>neg</code>	C3L excursion below C2L
<code>dist_mae_c2l</code>	<code>arr_c3h_vs_c2l</code>	<code>pos</code>	C3H rally above C2L
<code>dist_mfe_up_c2c</code>	<code>arr_c3h_vs_c2c</code>	<code>pos</code>	C3H above C2C
<code>dist_mfe_dn_c2c</code>	<code>arr_c3l_vs_c2c</code>	<code>neg</code>	C3L below C2C

Variable	Source Array	Filter	Meaning
<code>dist_mfe_up_c2o</code>	<code>arr_c3h_vs_c2o</code>	<code>pos</code>	C3H above C2O
<code>dist_mfe_dn_c2o</code>	<code>arr_c3l_vs_c2o</code>	<code>neg</code>	C3L below C2O

4.4 Two Contained Distributions (for MAE projection)

Variable	Source Array	Filter	Meaning
<code>dist_c3h_contained</code>	<code>arr_c3h_vs_c2h_contained</code>	<code>all</code>	Where C3H lands when it stays $\leq$ C2H
<code>dist_c3l_contained</code>	<code>arr_c3l_vs_c2l_contained</code>	<code>all</code>	Where C3L lands when it stays $\geq$ C2L

These are populated only from matches where C3 did NOT break the respective C2 level.

5. PATTERN MATCHING – PRESETS

5.1 Auto-Detect Presets

Preset	Dims	Dimensions Enabled
Minimal	2	C1 dir, C2 dir
Standard	5	+ C2H/C1H, C2L/C1L, C3O/C2O
Detailed	9	+ C2C/C1C, C2C/C1O, C2O/C1C, C2O/C1O, C3O/C2C
Full	16	All dimensions
Custom	varies	User picks via 16 individual checkboxes

Preset resolution: each `auto_*` boolean is set by a ternary – if Custom, use the `custom_*` input; otherwise, derive from preset level.

5.2 Manual Mode

Each of the 16 dimensions has a dropdown: "Any" / "Bull"or"Above" / "Bear"or"Below". "Any" means the dimension is not filtered on.

6. PROBABILITY-ADAPTIVE PROJECTIONS

6.1 Concept

The projected candle's high wick and low wick should reflect the most likely scenario:

- If C3H is likely to break above C2H → project using MFE (how far above C2H)
- If C3H is likely to stay below C2H → project using MAE/contained (where C3H typically lands when contained)
- Same logic for the low side

6.2 Mode Selection

User input per side: "Auto" / "MFE (Break)" / "MAE (Stay)"

Auto logic:

```
use_mfe_high = p_high_gt_c2h >= proj_auto_threshold  (default 50%)
use_mfe_low  = p_low_lt_c2l  >= proj_auto_threshold
```

6.3 Projected Candle Construction

High wick:

- MFE mode: $\text{proj_h} = \text{C2H} + (\text{C2C} \times \text{mfe_c2h_median} / 100)$
- MAE mode: $\text{proj_h} = \text{C2H} + (\text{C2C} \times \text{contained_h_median} / 100)$ (negative median → below C2H)
- Clamp: $\text{proj_h} = \max(\text{proj_h}, \text{c3o_price})$

Low wick:

- MFE mode: $\text{proj_l} = \text{C2L} + (\text{C2C} \times \text{mfe_c2l_median} / 100)$ (negative median → below C2L)
- MAE mode: $\text{proj_l} = \text{C2L} + (\text{C2C} \times \text{contained_l_median} / 100)$ (positive median → above C2L)
- Clamp: $\text{proj_l} = \min(\text{proj_l}, \text{c3o_price})$

Close:

- Primary: $\text{proj_c} = \text{C2C} + (\text{C2C} \times \text{med_c3c_vs_c2c} / 100)$ – anchored to where C3C typically lands relative to C2C
- Fallback: $\text{proj_c} = \text{c3o_price} + (\text{C2C} \times \text{med_body} / 100)$ – body size from open
- Clamp: $\text{proj_c} = \max(\text{proj_l}, \min(\text{proj_h}, \text{proj_c}))$

Body rendering:

- Open = c3o_price, Close = proj_c
- b_top = max(open, close), b_bot = min(open, close)
- Bull/bear color based on whether $\text{proj_c} \geq \text{c3o_price}$

6.4 Probability Zones

The zone boxes and median lines follow the same MFE/MAE adaptive logic:

- MFE mode: zones rendered in teal (high) / maroon (low), anchored above C2H / below C2L
 - MAE mode: zones rendered in orange, anchored near/below C2H / near/above C2L
 - Zone position: $\text{C2_level} + (\text{C2C} \times \text{zone_boundary} / 100)$
-

7. VISUAL ELEMENTS

7.1 Main Statistics Table

Position: configurable (default Top Right), size: configurable

Rows: 25 max

Content:

⚡ REAL-TIME (or 📊 HISTORICAL)

n=XXX

== Direction ==

Bull XX% Bear XX%

== C3H vs C2H ==

Above XX% Below XX%

MFE +X.XX% (or MAE -X.XX%)

== C3L vs C2L ==

Above XX% Below XX%

MFE -X.XX% (or MAE +X.XX%)

== C3 Close ==

>C2H XX% >C2O XX%

>C2C XX% >C2L XX%

Med C3C +X.XX%

== Body ==

Med +X.XX%

Color coding:

- Sample count: green (≥ 50), yellow (≥ 20), orange (< 20)
- Probabilities: green/red based on bull/bear or above/below
- Close probabilities: green if $\geq 50\%$, red if $< 50\%$
- MFE values: yellow; MAE values: orange

7.2 Distribution Table

Position: configurable (default Bottom Right), size: configurable

Rows: 80 max (to accommodate multiple distributions)

Each distribution section:

▲ MFE C3H;C2H n=XX			Med +X.XX%
+0.10→+0.25	14	36%	
+0.25→+0.40	5	13%	
...			
Zone +0.10→+0.40			62%

Display toggles for each of the 8 distributions (default: MFE from C2H and MFE from C2L enabled).

Bar chart: `make_bar()` produces 0-8 block characters proportional to count/max_count. **CRITICAL:** Uses `float(cnt)/float(max_cnt)` to avoid int truncation.

Color coding:

- MFE from C2H: teal, zone highlight lime
- MAE from C2H: orange, zone highlight  `#FF6600`
- MFE from C2L: maroon, zone highlight fuchsia
- MAE from C2L: purple, zone highlight  `#CC44FF`
- MFE from C2C up: green/lime
- MFE from C2C down: red/fuchsia
- MFE from C2O up: aqua/lime
- MFE from C2O down:  `#FF4466`/fuchsia

7.3 Pattern Table

Position: configurable (default Bottom Left), size: configurable (default Small)

Hidden by default – enable via settings.

Shows active preset name and dimension count, current C1/C2 pattern, HH/HL/LH/LL structure, C30 position, current projection mode per side, and match count.

7.4 Projected Candle

Rendered as box (body) + two lines (wicks) + label.

Positioned at `c2_bar + candle_offset`.

7.5 Reference Lines

Dotted lines at C2H, C2L, C2O, C2C (individually toggleable).

Default: C2H, C2L, C2C shown; C2O hidden.

7.6 Probability Zone Boxes

Shaded box from auto-zone lo→hi, with labeled percentage.

7.7 Median Lines

Solid lines at the median MFE or MAE level, with labeled value.

8. INPUT GROUPS AND DEFAULTS

Group	Input	Type	Default	Range/Options
General	Minimum Samples	int	1	min 1
	Analysis Mode	string	"Real-time (C3 Forming) "	Real-time / Historical
	Filter Mode	string	"Auto-Detect"	Auto-Detect / Manual
Auto-Detect	Pattern Preset	string	"Standard"	Minimal/Standard/Detailed/Full
Custom Dims	16 × bool	bool	varies	true/false
Manual Filters	16 × string	string	"Any"	Any/Bull-Bear or Any/Above/Bel
Historical Mode	Hypothetical C3 Open	string	"In C2 Body"	5 zones
Main Table	Show	bool	true	
	Position	string	"Top Right"	9 positions
	Size	string	"Normal"	Tiny/Small/Normal/Large
Pattern Table	Show	bool	false	
	Position	string	"Bottom Left"	9 positions
	Size	string	"Small"	Tiny/Small/Normal/Large
Distribution Table	Show	bool	true	
	Position	string	"Bottom Right"	9 positions
	Size	string	"Normal"	Tiny/Small/Normal/Large

Group	Input	Type	Default	Range/Options
	Number of Buckets	int	5	3-8
	Trim Percentile	int	5	0-20
	Auto-Zone Threshold	int	50	30-80
	Enable Auto-Zone	bool	true	
Dist Toggles	8 × show	bool	MFE C2H + MFE C2L on, rest off	
Projected Candle	Show	bool	true	
	Width	int	1	1-10
	Offset	int	5	1-20
	Bull Color	color	teal@30	
	Bear Color	color	maroon@30	
Prob Zones	Show Zones	bool	true	
	Show High/Low Zone	bool	true each	
	Show High/Low Median	bool	true each	
	Upside/Downside Color	color	teal@75 / maroon@75	
Projection Mode	High Projection	string	"Auto"	Auto/MFE/MAE
	Low Projection	string	"Auto"	Auto/MFE/MAE
	Auto Switch Threshold	int	50	30-70
Reference Lines	C2 High/Low/Open/Close	bool	H+L+C on, O off	

Group	Input	Type	Default	Range/Options
Labels	Size	string	"Normal"	Tiny/Small/Normal/Large
	Offset	int	8	1-50
	Line Length	int	6	1-50
	MFE Decimals	int	2	1-4

9. ALERTS

Alert	Condition
High Break	$P(C3H > C2H) > 65\%$ and $n \geq \text{min_samples}$
Low Break	$P(C3L < C2L) > 65\%$ and $n \geq \text{min_samples}$
Bullish	$P(\text{Bull}) > 60\%$ and $n \geq \text{min_samples}$
Bearish	$P(\text{Bull}) < 40\%$ and $n \geq \text{min_samples}$

10. DATA ARRAYS REFERENCE

Raw Excursion Arrays (populated in scan loop)

Array	Formula	Sign Expected
<code>arr_c3h_vs_c2h</code>	$(C3H - C2H) / C2C \times 100$	positive = broke above
<code>arr_c3l_vs_c2h</code>	$(C3L - C2H) / C2C \times 100$	negative = dropped below C2H
<code>arr_c3l_vs_c2l</code>	$(C3L - C2L) / C2C \times 100$	negative = broke below
<code>arr_c3h_vs_c2l</code>	$(C3H - C2L) / C2C \times 100$	positive = rallied above C2L
<code>arr_c3h_vs_c2c</code>	$(C3H - C2C) / C2C \times 100$	mostly positive
<code>arr_c3l_vs_c2c</code>	$(C3L - C2C) / C2C \times 100$	mostly negative
<code>arr_c3h_vs_c2o</code>	$(C3H - C2O) / C2C \times 100$	mostly positive

Array	Formula	Sign Expected
<code>arr_c3l_vs_c2o</code>	$(C3L - C2O) / C2C \times 100$	mostly negative
<code>arr_c3_body</code>	$(C3C - C3O) / C2C \times 100$	pos=bull, neg=bear
<code>arr_c3c_vs_c2c</code>	$(C3C - C2C) / C2C \times 100$	pos=close above C2C

Conditional Arrays

Array	Condition	Meaning
<code>arr_c3h_vs_c2h_contained</code>	$C3H \leq C2H$	How far below C2H was C3H when it didn't break up
<code>arr_c3l_vs_c2l_contained</code>	$C3L \geq C2L$	How far above C2L was C3L when it didn't break down

Normalization

All excursion values are normalized as a percentage of C2 Close:

```
excursion_pct = (price_diff) / C2C * 100
```

When converting back to price for chart rendering:

```
price_level = reference_level + (C2C * excursion_pct / 100)
```

11. KNOWN CONSTRAINTS AND PINE SCRIPT PITFALLS

11.1 Integer Division

Pine Script v6 performs integer division on `(int / int)`. ANY division involving two int variables that should produce a float MUST explicitly cast at least one operand:

```
pine
float result = float(numerator) / float(denominator) * 100.0
```

This applies to: probability calculations, zone percentages, bucket percentages, bar chart proportions.

11.2 `barstate.islast` Scanning

The full historical scan MUST run inside `barstate.islast`. Running it in the bar-by-bar loop causes the "current pattern" to change on each bar, making the filter compare each historical triplet against a different pattern.

The correct architecture:

1. **Every bar:** Push raw OHLC into storage arrays
2. **Last bar only:** Read current pattern → loop through all stored triplets → filter → accumulate

11.3 Array Memory

Storing 12 arrays of all historical OHLC uses significant memory. With `max_bars_back=5000`, this is ~60,000 float values. Pine Script's array limit is 100,000 elements per array, so this is well within bounds.

11.4 Loop Computation Time

The scan loop on `barstate.islast` iterates over all stored triplets (~5000 on daily data), and for each calls 16 filter checks. This runs once per bar update, which is fast enough. The distribution building adds another pass per enabled distribution, plus $O(n^2)$ for the auto-zone search across buckets (but buckets are capped at 8, so this is bounded).

11.5 Table Row Limits

Tables have a practical limit. Main table uses 25 rows, distribution table uses 80 rows. With all 8 distributions enabled at 8 buckets each, worst case is $\sim 8 \times (1 \text{ header} + 8 \text{ buckets} + 1 \text{ zone} + 1 \text{ separator}) = \sim 88$ rows. The 80-row allocation may need adjustment if all distributions are enabled simultaneously at max buckets.

12. VERSION HISTORY

Version	Key Changes
---------	-------------

v17.3	Full distribution table with clustering, dynamic bucketing, bar charts, auto-zone detection
--------------	---

Version Key Changes

v17.4 Added MAE distributions from all C2 OHLC levels. Fixed integer division bugs in all probability and percentage calculations. Restructured historical scan to run on `barstate.islast`. Added `DistResult` UDT and generic `build_dist()` function.

v17.5 Simplified Auto-Detect presets (Minimal/Standard/Detailed/Full/Custom). Probability-adaptive MFE/MAE projections with auto-switching. C3 Close probabilities vs all C2 OHLC. Projected candle close anchored to median C3C vs C2C. Pattern table hidden by default with configurable size.

13. FUTURE CONSIDERATIONS

- **Session filtering:** Match only triplets from the same trading session (Asia/London/NY)
- **Volatility normalization:** Normalize excursions by ATR instead of C2C for cross-timeframe comparison
- **Conditional close distributions:** C3C distribution split by whether C3 was bullish or bearish
- **Multi-timeframe:** Pull C1/C2 from a higher timeframe
- **Export/webhook:** Structured data output for external analytics
- **Distribution table for close:** Bucketed C3C vs C2C distribution (same engine, new toggle)
- **Performance optimization:** If array sizes grow large, consider downsampling or rolling windows